

imilar names, so be sure to pick the right one.

1. In the Actions section, require 1 approval from an individual user. Click your username as the approval user.

1. Leave the Override project approval settings checkboxes selected and click Configure with a Merge Request.

1. In the resulting Merge Request, click the Merge button.

1. When you create a policy, GitLab will create a new project for managing the policies. After you click Merge, you will need to navigate back to your original project.

Task D. View the License Compliance report for a pipeline

1. In the left sidebar, select Build > Pipelines.

1. Select the Run pipeline button at the top right of the page.

1. Leave all of the options at their default values, then select Run pipeline.

1. Monitor your pipeline and wait for each job to complete successfully.

1. Once all of the jobs are complete, refresh the page. Select the Licenses tab.

1. Notice that GitLab now describes every license except the MIT License as a policy violation. This report helps to flag existing dependencies which may contain licenses that you have not approved.

Task E. Create an MR that adds a dependency with a denied license

1. Create a new branch named add-dnspython-dependency.

1. Add a new dependency to the add-dnspython-dependency branch (not the main branch!) by pasting dnspython==2.1.0 as a new line at the end of requirements.txt.

1. Click the Create Merge Request button.

1. Leave all fields as their default values and click Create Merge Request.

1. Wait for the pipeline for the Merge Request to complete.

1. Once the pipeline completes, refresh the page. A license compliance scan will now display, showing 1 new license detected.

1. Return to the Merge Request overview.

1. Click Full report.

1. The license compliance report will list all of the licenses that are no longer in compliance

with the project policy. Note that the merge status will appear stating Requires 1 approval from policy.

1. A new comment is also added to the activity of the Merge Request, stating that a policy violation was detected.

> This Merge Request now requires additional approval because dnspython does not use an approved license. A reviewer can now verify if the license is ok to proceed with before the merge is complete.

Task F. Lab Policy Cleanup

> To ensure that you can merge branches into main without approval, it is best to disable the policy.

1. Navigate to Secure > Policies.
2. Click on the policy you created. Click Edit policy.
3. Click Delete Policy.
4. Click Merge on the resulting Merge Request.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the Hands-On Guide for GitLab Compliance, please submit them via Merge Request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabcompliancehandson5.md

> Estimated time to complete: 15 minutes

Objectives

To ensure that your CI/CD jobs run in a consistent and compliant way, you can enforce job configurations through pipeline execution policies. In this lab, you will practice enabling and configuring pipeline execution policies in your project.

Task A. Creating a standard pipeline

To start, we will create a standard pipeline to use in our compliance projects.

1. Navigate back to your ILT group.

1. Select New project.
1. Select Create blank project.
1. In the project name, enter CICD Template.
1. Leave all other options as default and select Create project.
1. Select + > New file.
1. In the Filename, input .gitlab-ci.yml.
1. In the file, add the following contents:

```
yml
stages:
  - test
```

```
include:
  component: ilt.gitlabtraining.cloud/components/sast/sast@main
```

1. Select Commit changes.

This CI/CD configuration contains a SAST scanner. By adding this in a pipeline execution policy, we can ensure that our project always runs a SAST scan. Let's see how to configure this for our project.

Task B. Adding a Pipeline Execution Policy

1. Return to your Compliance project.
1. In the left sidebar, select Secure > Policies.
1. Select New policy.
1. Under Pipeline execution policy, select Select policy.
1. Set the Name to Enforce Security.
1. Under Actions, select Inject from the CICD Template project.
1. Point the file path to your .gitlab-ci.yml file.
1. Leave all options as default and select Update via merge request.
1. Select Merge.
1. Return to the Compliance project project.

To test these changes, let's create a change in the project.

1. In the left sidebar, select Code > Repository.

1. Select + > New file.

1. In the Filename, input test.py. In the contents, add the following code:

```
python
import hashlib as h

h.md5('1')
```

1. Select Commit changes.

1. Select Create merge request.

1. Review the pipeline created from the merge request. Notice that it has a semgrep-sast job even though it's not defined in Compliance Project. This job comes from our pipeline execution policy. Note that we were able to add a job into the pipeline because we used the 'inject' keyword when setting up our policy. If we instead used the 'override' keyword, it would run this job instead of the jobs in your pipeline.

1. Return to your merge request. You will see that there is one new vulnerability, detected by the SAST scanner.

1. Select Merge to complete your merge request.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the Hands-On Guide for GitLab Compliance, please submit them via Merge Request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabcompliancehandson6.md

> Estimated time to complete: 15 minutes

Objectives

Learners will run a few actions in GitLab that show up in the Compliance Center and view how you could see a framework in action.

Task A. Creating Compliance Center Events

1. Navigate to your Compliance Project project.

1. In the left sidebar, select Secure > Compliance Center.

In this section, you will see three failed checks in your project. In cases where the Status is Fail, you will see View details (fix available) in the More information column. Let's see what information is shown in this section.

1. Select View details (fix available) in the At least one non-author approval row of your Compliance Center report.

1. Review the details outlined in this section.

1. Close the details panel.

Throughout the labs in this course, we will see how to fix each of these compliance issues.

Task B. Creating Compliance Center violations

In this task, we will complete some actions in our project that will result in Compliance Center violations.

1. In the left sidebar, select Code > Repository.

1. Select + > New file.

1. For the filename, enter main.py.

1. Add the following content to the file:

```
python
print("Start compliance project")
```

1. Underneath the Commit Message in the Target Branch field, enter new-main.

1. Ensure that Start a new merge request with these changes is selected.

1. Select Commit changes.

1. In the resulting merge request, select Assign to me in the Assignees section.

1. Select Create merge request.

1. Select Merge.

1. After the merge request completes, select Secure > Compliance Center.

1. Select the Violations tab.

1. Review the violation labelled Less than 2 approvers.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you wish to make a change to the Hands-On Guide for GitLab Compliance, please submit your changes via Merge Request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabcompliancehandson7.md

> Estimated time to complete: 15 minutes

Objectives

This lab will demonstrate how to view project and group level audit events.

Task A. Viewing Project Level Audit Events

1. Navigate to your Compliance Project project.

1. In the left sidebar, select Secure > Audit events.

1. Examine the types of Audit events that are captured in this report.

> Every audit event contains an author, object, action, target, IP address, and date.

1. Try the various filters in the Audit events report to see how you can filter data.

Task B. Viewing Group Level Audit Events

1. Navigate to your ILT group (titled My Test Group - random string of characters).

1. In the left sidebar, select Secure > Audit events.

1. Examine the types of Audit events that are captured in this report.

> Like the project audit report, every audit event contains an author, object, action, target, IP address, and date.

1. Try the various filters in the Audit events report to see how you can filter data.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the Hands-On Guide for GitLab Compliance, please submit them via Merge Request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabcompliancehandson8.md

> Estimated time to complete: 15 minutes

Objectives

This lab will outline how to access the various compliance reports, including audit reports and SBOM reports.

Task A. Viewing Compliance Reports and Permissions

1. Navigate to your project.

1. In the left sidebar, select Secure > Compliance center.

1. Review the contents of the report.

1. To view user permissions, navigate to Manage > Members.

1. Review the different users and roles in this section. Note that your user is the owner of the project, allowing you to view all details of all reports.

1. In the top right, you will see different options for adding users to your project. Explore each option and the different roles available in each option.

Task B. Viewing an SBOM Report

As a part of our compliance labs, we have been running a dependency scanner. This scanner produces an SBOM report when it runs. To view the SBOM report:

1. In the left sidebar, select Build > Pipelines.

1. Select the most recently passed pipeline.

1. Select the dependency-scanning job.

1. In the right sidebar, select Browse under Job artifacts.

1. Review the reports shown here. You can download these reports to view them in more detail.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the Hands-On Guide for GitLab Compliance, please submit them via Merge Request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabfundamentalshandson.md

GitLab Fundamentals Lab Guides

Lab Name	Lab Link
-----	-----
Organizing your work in GitLab	Lab Link
Project Management with Issues and Boards	Lab Link
Repository Management and Branching	Lab Link
Collaboration and Code Review	Lab Link
Continuous Integration and Development	Lab Link
Implementing Security Scanners	Lab Link
Viewing Analytics	Lab Link

Quick links

[GitLab Fundamentals course description](#)
[GitLab Certified Associate Certification Details](#)

Suggestions?

If you'd like to suggest changes to the GitLab Fundamentals Hands-on Guide, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabfundamentalshandsonlab1.md

> Estimated time to complete: 30 minutes

Objectives

In this lab, we will explore how we can organize work in GitLab, using groups, sub-groups, and projects.

Task A. Viewing a group structure

To start, in the left sidebar, select Groups.

1. In the Groups section, select the dropdown arrow on the left of Training Users.

1. Below Training users, you should now see a group with a name like Session <string of characters and numbers>.

1. Click the dropdown arrow next to the Session group. You should now see a group named My Test Group - <string of characters and numbers>. Click this group to access it.

This navigation shows an example of groups and subgroups. In the ILT GitLab instance, the Training Users group is a top-level group. This means that it is not part of any other groups in the instance. The Training Users group contains a set of subgroups, which represent classes running in the instance. Each of these subgroups is given a name like Session <string of characters and numbers>. Inside of these groups is a set of subgroups, one group for each student in the class. The My Test Group subgroup is your own group, which you can organize and modify in any way you like.

Task B. Creating a Group Structure

To start, make sure you are in the My Test Group subgroup. For this task, imagine you are working at a company with two departments: Development, and QA. These departments have the following projects:

- Development: A mobile and desktop version of an application named Cool App. Of note, Cool App contains many microservices spread over many repositories for both versions.
- QA: Two testing frameworks, one for the mobile version of Cool App and one for the desktop version. Both frameworks are monolithic, stored in a single project.

Take some time here to consider a structure for these two teams. After considering options, take a look at the steps below for a possible solution.

1. Since we have two teams, let's create a group for each team. Select New subgroup.

1. In the Subgroup name, enter Development. Leave all other options as default and select Create subgroup.

1. At the top of the page, you will see a set of breadcrumbs which shows your location in the application. Click My Test Group to return to your main group.

1. From here, you will now see your Development group. Select New subgroup to create your group for QA.

1. In the Subgroup name, enter QA. Leave all other options as default and select Create subgroup.

1. Select My Test Group from the breadcrumbs again to return to your main group. You will now see two groups: Development and QA.

Next, we can structure subgroups for both team's projects. Since the development applications

have many projects, it makes sense to create a subgroup for each project type.

1. Select the Development group.

1. Select New subgroup.

1. In Subgroup name, enter the name Cool App Desktop. Select Create subgroup.

1. In the breadcrumbs, select Development to return to the development group.

1. Select New subgroup.

1. In Subgroup name, enter the name Cool App Mobile. Select Create subgroup.

1. In the breadcrumbs, select Development to return to the development group.

Now in the Development group, you will see a Cool App Desktop and Cool App Mobile subgroups, ready to contain any projects related to both applications.

For the QA team, they have a single project. There are two main options for structuring this team. We can create a group for Cool App Testing that will contain the project, or we can just create the project in the subgroup.

Task C. Creating a repository

Let's create a new repository for the QA team. Select the QA group in your My Test Group. From here:

1. Select New project.

1. Select Create blank project.

1. In the Project name, enter Cool App QA.

1. Ensure that Initialize repository with a README is checked on. This will create a README file in the project as a starting point.

1. Select Create project.

After creating the project, you will be navigated to the repository of the project. You will see a single file titled README.md, which contains a basic README template. To get started working with this project, let's create an issue.

Task D. Creating issues in a project

Let's learn the basics of creating issues in GitLab.

1. Navigate to your Cool App QA project.

1. In the left sidebar, select Plan > Issues.

1. Select New issue.

1. In the title, enter First issue.

1. Enter any description you like.

1. Select Create issue.

By following this process, you created an issue related to your Cool App QA project. Let's discuss some of the details related to your issue. In the center, you will see your issue description, as well as activity for the issue and an area to add comments and discussions. Every issue can have child items, which are smaller tasks related to the issue. Issues can also be linked to other issues to show relationships between issues.

In the right sidebar, you will see metadata related to your issue. The Assignee is the set of people currently working on the issue. If the issue is associated with an epic, milestone, or iteration, these details will display here. Labels can be added to an issue to help organize issues into similar categories or types. Weight defines the amount of work associated with an issue. A due date can be set on an issue for when it should be completed by. Time tracking lets you track time taken to complete an issue. Health status lets you set a status for how the issue is progressing.

Often, we use issues to represent smaller pieces of work that may be completed in a single iteration or sprint. For larger sets of work, we use the concept of an epic, which will be discussed in a later lab.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the lab, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabfundamentalshandsonlab2.md

> Estimated time to complete: 15 minutes

Objectives

The goal of this lab is to get an overview of the different plan features available in GitLab, including epics and issue boards.

Task A. Defining an epic

An epic is created at a group level and defines larger sets of work on a project. To create an epic:

1. Navigate to your QA group.
1. In the left sidebar, select Plan > Epics.
1. Select New epic.
1. For the title, enter My first epic.
1. Enter any description you like and select Create epic.
1. Select My first epic.

In this view, you will see details on your epic. Similar to issues, epics can contain child items and linked items, as well as activity related to the epic. In the right sidebar, there is metadata for the epic, including labs, health status, parent, and time tracking.

Task C. Defining a milestone

A milestone represents a set of work being completed over a fixed time period. A milestone can contain both epics and issues. To create a milestone:

1. Navigate to your QA group.
1. In the left sidebar, select Plan > Milestones.
1. Select New milestone.
1. In the title, enter My first milestone.
1. Enter any Start Date and Due Date you would like.
1. Input any Description, then select Create milestone.

Next, we need to assign issues to the milestone.

1. In the left sidebar, select Plan > Issues.
1. Select First issue.
1. In the right sidebar, select Edit next to Milestone.
1. Select My first milestone from the resulting dropdown.
1. Select My first milestone in the right sidebar to be redirected back to the milestone.

You will now see all issues related to the milestone, along with burndown and burnup charts for the milestone dates provided.

Task D. Issue boards

To help with viewing issues and epics in a singular place, you can utilize issue and epic boards. To start:

1. Navigate to the QA group.

1. In the left sidebar, select Plan > Issue boards.

> In this view, you will see all your issues associated with their statuses.

1. We are going to create a new list to store our issues that are associated with us. Click the New List button on the right.

1. Select Assignee, then select your username.

1. Select Add to board.

1. You should see your issue you created before in the 'Open' list. Click and drag the issue from the 'Open' list to the list with your username.

1. Click on the issue. You should see that you have been automatically assigned to the issue.

It is possible to further customize issue boards to display different sets of information depending on your requirements. For more details on issue boards, check out the documentation.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the lab, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabfundamentalshandsonlab3.md

> Estimated time to complete: 15 minutes

Objectives

In this lab, we will explore how we can create repositories to organize code, as well as configure the repositories to meet various use cases.

Task A. Configuring your project

In the left sidebar, you will see a few new options available when you are in your project.

1. Navigate to your Cool App QA project.

1. In the left sidebar, select Settings. From the resulting dropdown, select General.

1. Review the different settings available in general. Here you can toggle your project visibility, features enabled, as well as advanced options like moving and deleting projects.

1. In the left sidebar, under Settings, select Repository. This section contains configurations for your project code repository.

In most settings, you will want to toggle a few settings for your repository. First, you will want to ensure your project has a consistent default branch. This branch will be the branch that merge requests are made against in your project. For this main branch, you will want to ensure it is protected and not able to be pushed directly to. Let's look at how to configure these options.

1. In Repository Settings, select Branch defaults to expand this section.

> Here, you will see that main is your default branch.

1. Next, select Branch rules.

1. Beside the main rules, select View details.

1. Explore the settings available here. You will see that by default, Allow force push is disabled for the main branch. You will also see that users with a Maintainer role can push and merge to main. Ideally, we want to prevent anyone from pushing into main directly.

1. To prevent pushes to main, select Edit in the Allowed to push and merge section.

1. Check the No one checkbox. Select Save changes.

With this setting changed, now no one can directly push to main. Let's test this out to see the result.

Task B. Testing pushes to main

1. In the left sidebar, select Code > Repository.

1. At the top of the repository section, select + > New file.

1. In the Filename, enter test.txt.

1. Enter any text into the file contents.

1. Select Commit changes.

1. In the New branch section, notice that there is a note stating "You don't have permission to commit to main". This is a direct result of our branch rules. To accommodate the change, this window has automatically created a new branch for us to use.

1. In New branch, change the name to test-commit.

1. Uncheck Create a merge request for this change.

1. Select Commit changes.

After committing the changes, you will now see the file available. This created a new branch in your project. To return to your main branch, select the dropdown that currently says test-commit. From this dropdown, select main to see your main branch again.

Now that we have a change in a branch, we can explore the concept of merge requests to merge the changes into main in the next lab.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the lab, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabfundamentalshandsonlab4.md

> Estimated time to complete: 30 minutes

Objectives

In this lab, we will explore the process of creating and merging a merge request

Task A. Creating a merge request

In the last lab, you created a new branch called test-commit. In this section, we will create a merge request to merge the changes from this branch into our main branch.

1. Navigate to your Cool App QA project.

1. In the left sidebar, select Code > Branches.

1. In the test-commit row, select New.

1. In the Title field, enter the title Merging new file to main.

1. Check the box Mark as draft. This will mark the Merge Request as a draft, and prevent it from being merged until the Draft status is removed.

1. In Description, enter any description you would like.

1. In Assignees, select Assign to me.

1. Leave all other options as default and select Create merge request.

After selecting Create merge request, you will be redirected to the merge request page. Let's explore this page in more detail.

Task B. Exploring the merge request

On the main merge request page, you will find four tabs available:

Overview, which shows an overview of the merge request, including approvals, merge request status, Activity, and a comment area to add comments to a merge request.

Commits, which shows all of the commits that are part of the current merge request.

Pipelines, which shows any CI/CD pipelines associated with a merge request.

Changes, which shows a differential of the changes associated with the merge request.

Return to the Overview tab. In this tab, there are a few important details to note:

1. In the right sidebar, you will see details about your merge request. The merge request is currently assigned to you, meaning you are the one currently working on the merge request contents.

The Reviewers section shows any reviewers that have been assigned to a merge request. Currently this is empty, since approval is optional.

Labels allows you to add organizational labels to a merge request to keep track of it in context of other related work.

Milestone allows you to associate a milestone to a merge request

Time Tracking lets you track time against a merge request.

Participants shows everyone who has commented or committed for a merge request.

In the center of the screen, you will see a message stating Merge blocked. In this section, you can see any issues preventing your code from being merged into main. Anything from failed pipelines to security scan results can block a merge request, depending on your configuration. Currently, the reason the request is blocked is stated below: "Merge request must not be a draft". Let's fix this issue.

1. Click Mark as ready in the Merge blocked block. If you do not see the Mark as ready option, click on the arrow to the right of the Merge blocked block to expand it.

1. Select Merge.

1. Once the merge completes, in the left sidebar, select Code > Repository.

You will now see your new file in the main branch of your code repository.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the lab, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabfundamentalshandonlab5.md

> Estimated time to complete: 30 minutes

Objectives

In this lab, we will explore creating a basic CI/CD pipeline for our QA project.

Task A. Setting up a code base

Before we start creating a CI/CD process, we need some code to run our CI/CD process against. To start, navigate to your Cool App QA project.

1. In your project, select + > New file.
1. In the Filename field, select main.go.
1. Inside of main.go, add the following code:

```
go
package main

import(
    "fmt"
)

func main() {
    fmt.Println("We are up and running!")
}
```

1. Select Commit changes.
1. Click the radio button Commit to a new branch. This will give you an option to be able to create a new branch and merge request. In the Branch name, enter initial-code. Leave Create a merge request for this change checked. Select Commit changes.
1. Leave all the merge request options at their defaults and select Create merge request.

From here, we have one additional file to add to our code, which is a go.mod file. To add this:

1. In the left sidebar, select Code > Branches.

1. Select initial-code.

1. Select + > New file.

1. In the Filename field, type go.mod.

1. Add the following code to the file:

```
go
module array

go 1.22.2
```

1. Select Commit changes.

1. Ensure that Commit to current initial-code branch is selected. Select Commit changes.

With our code created, we can now start to create a CI/CD process for the code.

Task B. Creating a CI/CD Process

Let's create a CI/CD process for the code we just wrote. Our goal is to create a process that builds the code we wrote. To do this, we need to create a .gitlab-ci.yml file. This file will contain all jobs and stages for our CI/CD process.

1. In the left sidebar, select Code > Repository.

1. Select + > New file

1. In the Filename, input .gitlab-ci.yml.

1. Copy the following code into your .gitlab-ci.yml file:

```
yaml
default:
  image: golang

stages:
  - build

build go:
  stage: build
  script:
    - go build
```

> Every GitLab CI/CD job on this instance runs in a Docker container. The default line defines the Docker image to use to run the jobs for this `.gitlab-ci.yml` file. Below this, we defined one stage, which is `build`. In this stage, there is a single job, which runs one script: `go build`. The result of this will be your Go application being compiled.

1. Select Commit changes. Ensure that Commit to the current initial-code branch is selected.

1. Select Commit changes.

Task C. Viewing the CI/CD Process

1. After committing your code, your pipeline will immediately start. To view the pipeline, navigate to Build > Pipelines.

Here, you will see a summary of all of your project pipelines. Each pipeline shows the following details:

- The status of the pipeline
- The pipeline name, ID, branch, and triggering commit
- Who created the pipeline
- A breakdown of pipeline status by stage

1. To view more details about the pipeline, select the Status of the pipeline. In this UI, you will see a graph of the pipeline, showing each stage, and the jobs associated with the stage.

1. Select your build go job.

> On this screen, you will see details about your job, including all of the commands run during your job execution. On the right, you will see the duration of the job, when the job finished, how long the job was queued, the runner that completed the job, the commit that triggered the job, and further pipeline details related to the job.

Let's explore each of these in detail. To start, navigate to your job:

1. Select Build > Jobs.

1. Select your build go job.

Let's walk through the job log to better understand each job stage. The first thing you will see is something like this:

Setting up your job environment

```
bash
```

```
Running with gitlab-runner 17.0.0~pre.88.g761ae5dd (761ae5dd)
```

```
on green-6.saas-linux-small-amd64.runners-manager.gitlab.com/default YKxHNYexq, system ID: sa201ab37b78a
```

```
Resolving secrets
```

```
Preparing the "docker+machine" executor
```

```
00:19
```

```
Using Docker executor with image golang ...
Using docker image sha256:5905f95343e84d1f8f14aff8f8b83747fb39ea0e0fad52a9d14cf41860295fff for
golang with digest
golang@sha256:f43c6f049f04cbbbaeb28f0aad3eea15274a7d0a7899a617d0037aec48d7ab010 ...
Preparing environment
00:06
Running on runner-ykxhnyexq-project-58378461-concurrent-0 via runner-ykxhnyexq-s-l-s-
amd64-1717165680-d1e5066e...
```

The GitLab lab environment uses runner managers to help with scaling jobs. When your job starts, it first enters a queue. When a runner manager is available, it picks up the job. It then creates an instance and sets it up with the defined Docker image, in this case, the golang image. This image is pulled and loaded onto the runner, making it ready to start processing your job request.

Cloning your Git repository

After the environment setup, GitLab will clone your repository onto the runner.

```
bash
Getting source from Git repository
00:01
Fetching changes with git depth set to 20...
Initialized empty Git repository in /builds/scottcosentinogitlab/cicdlabrewrite/.git/
Created fresh repository.
Checking out 4ae4ca35 as detached HEAD (ref is main)...
Skipping Git submodules setup
$ git remote set-url origin "${CIREPOSITORYURL}"
```

After doing this, all of your code will be available on the runner. One important note is that your runner now has access to your Git repository and has a link to your remote repository. This means two things:

- You can access and use any files in your Git repository
- You can commit changes back to your repository if you make any during your job process

Optional Task:

Want to see this in action? Add the `ls` command to your job scripts. This will list the current directory, showing you all the files that were cloned to the runner.

```
yaml
default:
  image: golang
```

```
stages:
  - build
```

```
build go:
  stage: build
```

script:

- ls
- go build

Executing your Scripts:

After the environment is set up and your repository is cloned, your job scripts will run.

bash

Executing "steps script" stage of the job script

Using docker image sha256:5905f95343e84d1f8f14aff8f8b83747fb39ea0e0fad52a9d14cf41860295fff for golang with digest

golang@sha256:f43c6f049f04cbbbaeb28f0aad3eea15274a7d0a7899a617d0037aec48d7ab010 ...

\$ go build

Cleaning up project directory and file based variables

Job succeeded

To summarize, there are a few important ideas to keep in mind when considering running jobs in your pipeline.

- Jobs will generally use a Docker image to run your job scripts
- Every job runs on a separate runner, within its own Docker container, so there are no concerns about jobs interfering with each other
- You have full access to your Git repository and any other system resources during the execution of your jobs

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the lab, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabfundamentalshandsonlab6.md

> Estimated time to complete: 30 minutes

Objectives

This lab uses SAST, an optional feature in CI/CD pipelines, to identify security vulnerabilities in your code. GitLab's Vulnerability Report then shows any old or new vulnerabilities found with each pipeline run. You can learn more in the documentation.

Task A. Finding the SAST Component

GitLab provides a variety of CI/CD components, which are prebuilt CI/CD configurations you can include in your projects. To view all of the available components on your GitLab instance:

1. In the left sidebar, select Search or go to.

1. In the resulting dialog, select Explore.

1. In the left sidebar, select CI/CD Catalog. This will show you a list of all of the CI/CD catalog items available in your GitLab instance. For this lab, you will be adding SAST to your project. Select the SAST component.

When you select a CI/CD component, you will see a Readme, which describes how to use the component, as well as configuration options for the component. For the SAST component, you will see that it can be included using the following code:

```
yaml
include:
  - component: ilt.gitlabtraining.cloud/components/sast/sast@1.0.0
```

Let's add this to our CI/CD file.

Task B. Adding the SAST component

1. Navigate to your CI/CD project by clicking on the Tanuki logo in the top left corner of the page, then click on your Cool App QA project.

1. In the left sidebar, select Code > Branches.

1. Select the initial-code branch.

1. Select your .gitlab-ci.yml file.

1. Select Edit > Edit in Pipeline Editor.

1. At the top of your file, below the image, add the SAST import at version main.

```
yaml
include:
  - component: ilt.gitlabtraining.cloud/components/sast/sast@main
```

1. In the stages section of the file, add a stage named test, as shown below:

```
yaml
stages:
  - build
  - test
```

After making these changes, your file will look like this:

```
yaml
include:
  - component: ilt.gitlabtraining.cloud/components/sast/sast@main

default:
  image: golang

stages:
  - build
  - test

build go:
  stage: build
  script:
    - go build
```

1. Ensure that your Branch is set to initial-code. Select Commit changes.
1. After committing your changes, in the left sidebar, navigate to the Build > Pipelines.
1. Select the most recent pipeline.
1. You will now see a new job named semgrep-sast. This job is the security scan imported using the include keyword.

Task C. Add run.py and review SAST scanning results

In this task, you'll add a file with known vulnerabilities and see if SAST detects it.

1. Return to the Project overview page by clicking on the name of your project in the breadcrumbs section.
1. At the top of the project landing page, to the right of the branch dropdown, click (+) > This directory > New file.
1. For the File name field, type in run.py.
1. Copy the content below into the file:

```
python
import subprocess

in = input("Enter your server ip: ")
subprocess.run(["ping", in])

print("Attempting to connect to the server")
```

```
print("Application authentication was successful")
```

1. Add an appropriate Commit message and click the Commit changes button. Merge the code on the initial-code branch.

1. In the left-hand navigation pane, click Build > Pipelines.

1. At the top of the row of the table of pipelines, click on the running (if it is still running) or passed (if the pipeline has been completed) status labels.

> The SAST scan may take a few moments, so feel free to grab a cup of coffee while you wait.

1. Once the SAST scan has completed, in the left-hand pane, refresh the page, and click on the new tab in the pipeline details page called Security.

1. Click on the vulnerability listed and read about a potential security problem detected by SAST scanning in run.py.

1. Feel free to edit the code to fix the issue raised (such as removing the subprocess.run command), and commit the changes. Does the vulnerability report still note the issue as present?

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the lab, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabfundamentalshandsonlab7.md

> Estimated time to complete: 30 minutes

Objectives

The goal of this lab is to get an overview of the different analytics available to you in GitLab.

Task A. Viewing your analytics

To start, navigate to your Cool App QA project.

1. In the left sidebar, select Analyze > Analytics Dashboard.

1. Select Value Streams Dashboard.

In this report, you will see a variety of analytic details about your current project. Some details you may see populated here after the previous labs are: Merge request throughput, and Median time to merge.

1. In the left sidebar, select Analyze > CI/CD Analytics.

In this report, you will see details about your CI/CD pipelines. Here you can see how many pipelines have run in your project, the duration of the pipelines, and failure rates of pipelines.

1. Take some time to explore the other analytics reports available. Review any insights that seem interesting!

It's important to note here that you can also view analytics at a group level. To do this, navigate to your QA group. In the left sidebar, you can select Analyze and see similar reports for everything contained in a group.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the lab, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabpmhandson.md

GitLab Agile Portfolio Management Lab Guides

Lab Name	Lab Link
-----	-----
Access the GitLab training environment	Lab Link
Create an organizational structure in GitLab	Lab Link
Use GitLab planning tools	Lab Link
Create issues	Lab Link
Organize and manage issues	Lab Link
Use a merge request to review and merge code	Lab Link
Create and customize issue boards	Lab Link
Create and manage a Kanban board	Lab Link
Create and manage a Scrum board	Lab Link
Create and manage a Waterfall board	Lab Link

Quick links

Here are some quick links that may be useful when reviewing this Hands On Guide.

Suggestions?

If you'd like to suggest changes to the GitLab Agile Portfolio Management Hands-on Guide, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabpmhandsonlab1.md

> Estimated time to complete: 5 to 10 minutes

Objectives

In this lab, you will access the training lab environment. This environment will allow you to try all of the GitLab Ultimate features, providing you with an environment to follow along with all of the lab steps.

Task A. Access your Training Group and Lab Environment

1. Navigate to <https://gitlabdemo.com/invite> in a web browser.

1. In the Invitation Code field, enter the invitation code provided by your instructor or in the LevelUp LMS.

1. Select Provision Training Environment.

1. On the confirmation page, locate the Your GitLab Credentials section. Read this section carefully, noting the credential information provided and the expiration date. Your access to this group and all of the projects that you create is ephemeral and will be deleted after the expiration date.

1. Click the Download Credentials button to download your temporary GitLab credentials.

1. Click on GitLab Dashboard or the GitLab URL.

1. Login with your temporary GitLab Credentials.

> This group your temporary account has been created in has a GitLab Ultimate license to see all of the features available.

Suggestions?

If you'd like to suggest changes to this lab, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabpmhandsonlab10.md

> Estimated time to complete: 30 minutes

Objectives

The Kanban and Scrum boards in previous labs showed the progress of issues as developers worked on them. A waterfall board shows the health of all the issues that your team is responsible for within in a single waterfall stage. With a pure waterfall workflow, all issues are due at the same time--at the end of the waterfall stage--so what you're tracking with the board is how likely they are to be done by that date.

Task A. Make a milestone representing a waterfall stage

1. Navigate to the Awesome Software project.
 1. In the left pane, click Plan > Milestones.
 1. Click New milestone.
 1. Type Release 2.0-Dev in the Title field.
 1. Select today's date as the Start Date by clicking on today's date in the calendar.
 1. Select 1 month from today's date as the Due Date by clicking on today's date in the calendar.
 1. Write up a clear description for the milestone.
 1. Click Create milestone.

Task B. Add issues to the milestone

1. In the left pane, click Issues.
 1. Click Bulk Edit.
 1. Select a few issues of your choice by clicking the faint checkbox to the left of the search bar above the list of issues.
 1. In the right pane, select Release 2.0-Dev from the Milestone dropdown.
 1. At the top of the right pane, click Update all to apply the label to all selected issues.

Task C. Simulate past work in the waterfall stage by setting issue health statuses

1. For each issue in the milestone, choose one of the scoped labels to apply: Health::On Track, Health::Needs Attention, or Health::At Risk.

> Remember, to apply a label to an issue, click on the issue. Then, on the right side of the screen next to the Labels keyword, click on Edit. From there, apply one of the labels by clicking on the label name, and then clicking off to set it.

Task D. Make a board to track the health of issues in a single waterfall stage

1. In the left pane, click Plan > Issue Boards.

1. Using the dropdown at the top of the page, create a new board.

1. Title the board Release 2.0 Dev Stage

1. Deselect the Show the open list checkbox.

1. Leave the Show the Closed list checkbox selected.

1. Scope the board to display issues in the Release 2.0-Dev milestone.

1. Click Create board.

> We will now create three new lists for the board: a list with a Health::On Track label, a list with a Health::Needs Attention label, and a list with a Health::At Risk label.

1. In the top right corner, click Create list.

1. Check that the Label radio button is selected in the Scope section of the list configuration pane.

1. Open the Value drop-down, and select Health::On Track.

1. Click Add to board.

1. In the top right corner, click Create list.

1. Check that the Label radio button is selected in the Scope section of the list configuration pane.

1. Open the Value drop-down, and select Health::Needs Attention.

1. Click Add to board.

1. In the top right corner, click Create list.

1. Check that the Label radio button is selected in the Scope section of the list configuration pane.

1. Open the Value drop-down, and select Health::At Risk.

1. Click Add to board.

> At this point, you have a complete board set up, but it is still recommended that you practice using this board as you would in production. For example, practice dragging and dropping your issues across different lists as their at-risk status increases or decreases. Or, simulate completing some issues by dragging them into the Closed list. Make sure to click on the issues themselves to confirm they are closed.

Suggestions?

If you'd like to suggest changes, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabpmhandsonlab2.md

> Estimated time to complete: 30 minutes

Objectives

In GitLab, projects and subgroups help you organize your codebases and effectively manage your projects. In this lab, you will learn how to create an organizational subgroup, how to create a project, and how to add a user to a project.

Organization Namespaces: You can create subgroups under any other groups that you have access to and have permissions to create groups underneath. Your organization should have best practices documented on how groups are structured or named. If you're not sure, you can locate another group's members and ask one of the users with Owner role who should be able to help.

Personal Namespaces: Although you can create projects in your personal namespace (ex. <https://gitlab.com/exampleuser/example-project>), you cannot create groups in your personal namespace.

Security Warning: Any users that have access to parent or grandparent groups will have inherited permissions to this group. In the training environment, each student only has direct member access to their subgroup and does not have inherited member access to the parent groups. See the subgroup membership docs to learn more.

> Tip: Remember that the breadcrumbs are a good way to navigate between subgroups when you're creating a multi-level collection of groups and subgroups.

Prerequisites

1. You have already redeemed the invitation/voucher code.

1. You have signed into <https://ilt.gitlabtraining.cloud> with the username provided.

1. You have clicked the My Group button and opened a new browser tab with the URL <https://ilt.gitlabtraining.cloud/training-users/session-{id}/iu{iu}>. You should see My Test Group - iu{iu} as the title of the group.

Task A. Review groups and projects for GitLab's own source code

1. In a new browser tab, navigate to <https://gitlab.com/gitlab-org>. This group contains all of GitLab's product source code that is mostly open source and transparent based on our company values.
2. In the list of groups and project, click the linked name for the Frontend subgroup. You might have to scroll down or use pagination to find it on the second page.
3. Within the Frontend subgroup, you can see all of the projects and subgroups that exist as part of this group. Spend a few moments exploring some of these subgroups and projects.

Task B. Create organizational subgroups

1. Navigate to your personal top-level group by going to <https://ilt.gitlabtraining.cloud/training-users/session-<INVITATIONCODE>/<USERNAME>>, replacing the <INVITATIONCODE> with your provided code, and <USERNAME> with your username. If you are unsure of what your username is, click on the profile picture in the top right corner, and copy the username provided without the @ symbol.

1. Click the New Subgroup button in the top right corner.

1. In the Subgroup name field, type Awesome Inc.

1. The Subgroup URL shows which namespace hierarchy that this group will be created in.

- Do not change the Subgroup URL in this class. You will always create projects under <https://ilt.gitlabtraining.cloud/training-users/session-<INVITATIONCODE>/<USERNAME>> since this is your top-level organization group for these steps.

1. The Subgroup slug will automatically populate based on the name. Rename awesome-inc to awesome.

- Best Practice: This slug is what will appear as the project path, which means that it is both the URL path and the folder name when you run git clone. It is a best practice to shorten this when appropriate.

1. Ensure that the Visibility level is set to Private.

- > A private subgroup will ensure that the group and its projects can only be viewed by members. For more details of visibility levels, see the documentation.

1. Click the Create subgroup button.

- You should now see the group with the title Awesome Inc and your browser URL will reflect the path at <https://ilt.gitlabtraining.cloud/training-users/session-<INVITATIONCODE>/<USERNAME>/awesome>.

1. Congratulations! You have created your first group and are ready to create additional groups on your own. Within the Awesome Inc group, create subgroups for each of these teams at your awesome company.

- Software
 - Core (this is a subgroup of Software, not Awesome Inc)
 - Android
 - iOS
- Infrastructure
- Security

Task C. Create a new project

1. Navigate to the Awesome Inc > Software > Core subgroup that you just created.
1. Click the Create new project tile.
1. Click the Create blank project tile.
1. In the Project name field, enter Family Budget Calculator.
1. Note that the project URL shows /awesome/software/core in its path. This shows that the project is in the core subgroup.
1. Leave the project slug as the default value, family-budget-calculator.
1. Set the Visibility Level to private.
 - > See the visibility docs to learn more. Keep in mind that any parent groups that have Private visibility will be inherited for all child groups and projects and cannot be changed.
 - >
 - > Training Environment Limitation: In the training environment, your group is part of a private parent group so you will not be able to make a group or project Public. To test working with public projects, you can create a project in your personal namespace.
1. Enable the Initialize repository with a README checkbox.
1. Leave Enable Static Application Security Testing (SAST) unchecked.
1. Select Create project.

Task D. Add a project member and set their role

1. In the Family Budget Calculator project, click Manage > Members in the left sidebar.
1. Click the Invite members button in the top right corner.
1. Search for and select your instructor as the user you are inviting.
1. In the Select a role dropdown, select Developer.
 - > See the permissions and roles docs to learn more about each role level.
1. Click the Invite button.

1. Refresh the page to see the user invited as a Direct Member.

> Users will inherit permissions from the parent group hierarchy that this project exists in. If a user already has a higher level of access in a parent group (ex. Maintainer), that access level supersedes a lower level of permission assigned at the project level (ex. Developer).

Suggestions?

If you'd like to suggest changes, please submit them using a merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabpmhandsonlab3.md

> Estimated time to complete: 45-60 minutes

Objectives

To help you organize your work, GitLab provides epics, iterations, and milestones. In this lab, you will learn how to create, configure, and manage epics, iterations, and milestones in your projects.

Task A. Review planning features in GitLab

1. In a new browser tab, navigate to the GitLab project source code at <https://gitlab.com/gitlab-org/gitlab>. Note the namespace structure indicated by the top of the page. You should be in the GitLab project inside the GitLab.org group.

1. At the top of the page above the project name, click GitLab.org to navigate to the parent group.

1. Note the number of epics, issues, and merge requests indicated in the left pane. These numbers represent work items across all subgroups and projects inside Gitlab.org.

1. In the left pane, click Epics. This takes you to a searchable list of epics in GitLab.org and all its subgroups.

1. In the left pane, click Plan > Roadmap. Epics and milestones in a group containing a start date or due date can be visualized in a form of a timeline (that is, a Gantt chart). The Roadmap page shows the epics and milestones in a group, one of its subgroups, or a project in one of the groups.

1. On the epic bars, you can see each epic's title, progress, and completed weight percentage. When you hover over an epic bar, a popover appears with the epic's title, start date, due date, and weight completed.

1. You can expand epics that contain child epics to show their child epics in the roadmap. You can select the chevron (v) next to the epic title to expand and collapse the child epics.

Task B. Create an epic

> Epics provide a way to organize and manage a set of issues and sub-epics that share a strategic theme. In addition to logical grouping, epics enable project managers to perform higher level planning and build a roadmap with visual status tracking. You can read more about epics in the documentation.

1. In the lab environment browser tab, at the top of the page, click the ... beside GitLab Learn Labs.

1. Click on My Test Group.

1. On the groups page, click on the Awesome Inc subgroup. Inside the subgroup, click Software.

1. In the left pane of the Software group landing page, click Epics.

1. In the upper-right corner above the search bar, click on the New epic button.

1. On the New Epic page, enter Feature Category: Retirement Planning in the Title (required) field. Having a descriptive title helps make it clear what issues and sub-epics should be associated with the epic.

1. In the Description field, paste the following:

```
markdown
Overview
```

This is the top-level epic for all features in the Retirement Planning category of Awesome Co.'s personal finance software.

Useful Links

- To-Do: add link to the feature strategy document
- To-Do: add contributing team member information

> There is no need to change any other options for the epic, but it is still important to understand the options.

1. There are additional options that you can configure or leave at their default setting.

- Confidentiality: Create private epics for internal use if you have a public facing project.

- Labels: Apply labels to your epic, which are metadata tags that can be used to sort and filter your epics.

- Dates: Schedule your epics to be at a fixed start and due date, or have the epics inherit their state and end dates from the issues associated with them. You can also let the epic progress over time, without a fixed end date.

- Color: Customize the display color of the epic when used in roadmap timeline bars. Look at the GitLab.org group's Roadmap in Task A for an example.

- Parent: This enables you to list another epic as a parent of this epic. This helps you keep your hierarchy clear and makes it easy to see which epics are related.

Task C: Create a child epic

> A parent epic can have multiple child epics. Use child epics to organize more complex topics into smaller, more focused epics and issues.

1. You will now create a new child epic to link to the parent epic. Navigate to your Epics page by clicking on Plan > Epics on the left pane.

1. Click the New epic button in the top right corner.

1. In the title section, type in Investment Tracking.

1. In the Description field, paste the following:

```
markdown
Overview
```

This epic tracks all work on Investment Tracking features and integrations, as part of the overall Retirement Planning category strategy.

1. Leave all other fields as they are, and click Create epic.

1. Return to the Software group's full list of epics by clicking Epics in the breadcrumbs at the top of the page.

1. Click into the Feature Category: Retirement Planning epic.

1. You will now designate the Investment Tracking epic as a child of the Retirement Planning epic. In the Child issues and epics tab, select the Add drop-down menu, and click Add an existing epic.

> You can also use this menu to create a child epic from scratch.

1. Type & in the field provided, and select Investment Tracking from the list of epics.

> The & symbol lets you refer to epics by their unique epic ID rather than their name. This is useful for when you have a large number of epics with similar names.

1. Click Add to link Investment Tracking as a child epic to the Retirement Planning parent epic.

Task D. Set milestones to represent product goals

> Milestones in GitLab are a way to track issues and merge requests created to achieve a broader goal in a certain period of time. Milestones allow you to organize epics, issues and merge requests into a cohesive group, with an optional start date and an optional due date. You can read more about milestones in the documentation.

1. Navigate to the Awesome Inc subgroup.

1. In the left pane, click Plan > Milestones. This will take you to the Milestones page.

1. Click New milestone in the top right corner.

1. In the Title section, type in Organization Kickoff.

1. In the Start Date section, use the calendar to enter in today's date.

1. In the End Date section, use the calendar to enter 2 days from today's date as the milestone end date.

> While a description is not necessary, it is recommended to type in a description to help clarify what the milestone is intended to be used for.

1. Click Create milestone.

1. In the breadcrumbs at the top of the page, click Milestones.

1. Click New milestone to create a second milestone.

1. In the Title section, type in Backend services deployed.

1. In the Start Date section, use the calendar to enter in today's date.

1. In the End Date section, use the calendar to enter 2 weeks from today as the milestone end date.

1. Click Create milestone.

1. In the breadcrumbs at the top of the page, click Milestones to view your newly created milestones.

You will later assign tasks to the epics and milestones you created, allowing you to use roadmaps to view the progress of your initiatives.

Task E. Schedule iterations as team sprints

> Iterations are mutually exclusive timeboxes intended to track team velocity, while milestones can represent larger, overlapping product goals. You can read more about iterations in the documentation.

1. In your subgroup structure, navigate to Awesome Inc > Software.

1. In the left pane, click Plan > Iterations.

1. Click New iteration cadence in the top right corner.

1. In the Title section, type in Team sprints .

1. In the Description section, type in Tracking team progress toward minimum viable product .

1. In the Automatic start date input, use the calendar to enter in today's date.

1. In the Duration section, select 2 weeks as the duration of each iteration.

1. In the Upcoming iterations section, select 6 for the number of upcoming iterations.

1. Ensure the Enable roll over checkbox is selected.

> At the end of the current iteration, all open issues are added to the next iteration when Enable roll over is selected.

1. Select Create cadence.

You will later assign individual tasks to iterations (sprints).

Task F. Create a wiki for project documentation

> If you don't want to keep your documentation in your repository, but you want to keep it in the same project as your code, you can use the wiki GitLab provides in each GitLab project. Every wiki is a separate Git repository, so you can create wiki pages in the web interface, or locally using Git. GitLab wikis support Markdown, Rdoc, AsciiDoc, and Org for content. Wiki pages written in Markdown support all Markdown features, and also provide some wiki-specific behavior for links. You can read more about wikis [here](#).

1. Navigate to your Family Budget Calculator project in the Software > Core subgroup.

1. In the left pane, click Plan > Wiki.

1. Click Create your first page button in the middle of the page.

1. In the Title section, type in Family Budget Calculator Documentation as the page title.

1. Paste the following text in the content field underneath the title field.

markdown
Summary

The Family Budget Calculator helps households stay on budget and save for the future.

Contact

Contact <YOUR-NAME> with questions or comments.

1. Click Create Page. You should see a new Wiki page with the content you created.

> If you'd like, edit the Family Budget Calculator Documentation page to add additional content or create additional wiki pages.

Suggestions?

If you'd like to suggest changes, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabpmhandsonlab4.md

> Estimated time to complete: 45 minutes

Objectives

Issues are a core building block in GitLab that enable collaboration, discussions, planning and tracking of work. Issues are defined in the scope of a Project, not a Group. You can read more about them in the documentation.

Task A. Create and manage labels

> You can use labels to categorize epics, issues, and merge requests using colors and descriptive titles like bug, feature request, or docs. This allows you to dynamically filter and manage epics, issues, and merge requests. You can read more about them in the documentation.

1. Navigate to your Awesome Inc group.

1. In the left pane, click Manage > Labels.

1. Click New label in the upper right hand corner.

1. In the Title field, enter Status::Open. The 2 colons in the label title mean that this will be a scoped label.

> A scoped label uses a double-colon (::) syntax in its title, for example: workflow::in-review. An issue, merge request, or epic cannot have two scoped labels, of the form key::value, with the same key. If you add a new label with the same key but a different value, the previous key label is replaced with the new label.

1. In the Description field, type Item that is ready to begin work .

1. For the label's color, GitLab supports any hex color code. For this label, choose Blue-gray from the suggested color palette (or type 6699cc in the Background color field).

1. Click Create label.

1. Create the following additional labels, setting a description and background color of your choosing. Note that some of these are scoped and some are unscoped.

- Status::Open
- Status::WIP
- Status::Done
- Priority::High
- Priority::Medium
- Priority::Low
- Dev
- QA
- Security

1. Go to your Family Budget Calculator project. The project is inside the Awesome Inc > Software > Core group hierarchy.

1. Click Manage > Labels from the left sidebar.

1. Click the star icon to the left of the Subscribe button to designate the following labels as prioritized labels. Prioritized labels appear at the top of your labels list.

- Priority::High
- Priority::Medium
- Priority::Low

Task B. Create issues for tracking work

1. In your Family Budget Calculator project, click Plan > Issues from the left sidebar.

1. Click the New issue button.

1. In the title section, type Third-party financial services integration.

1. The description is an optional section, but feel free to type in your own description for this issue.

1. Using the Assignees dropdown, assign the issue to yourself by clicking on the dropdown, and then clicking on your username. While we will leave the options as they are for now, it is important to understand what they do:

- Epic: Associates the issue with an epic.

- Labels: Apply labels to your issue, which are metadata tags that can be used to sort and filter your issues.

- Iteration: Associate the issue with an iteration to track it over a period of time. This allows teams to track velocity and volatility metrics.

- Milestone: Milestones in GitLab are a way to track issues and merge requests created to achieve a broader goal in a c